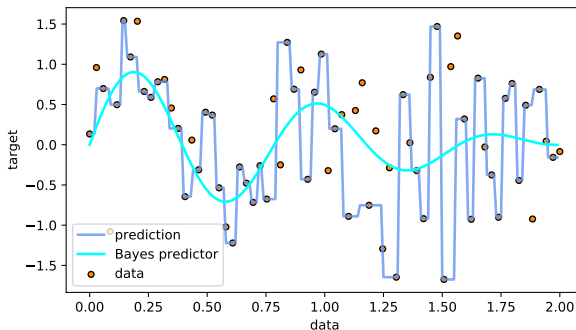


Machine learning I, supervised learning: classification and regression trees

Bagging regression
number of estimators: 1
test error: 1.22E+00
Bayes risk: 7.00E-01



Decision trees

- Decision trees

- Construction of a decision tree

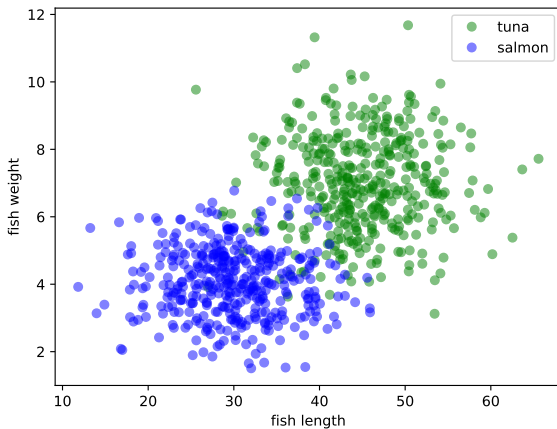
- Tree pruning

Ensemble learning

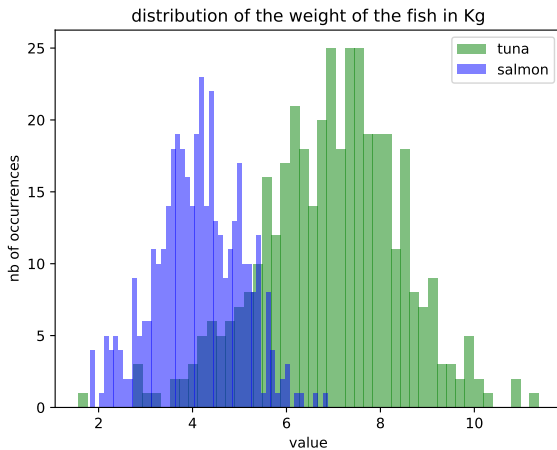
- Bagging

- Random forest

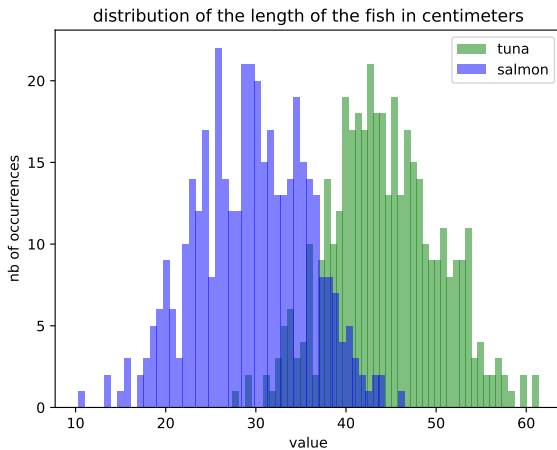
Example dataset : fishes



Fish dataset



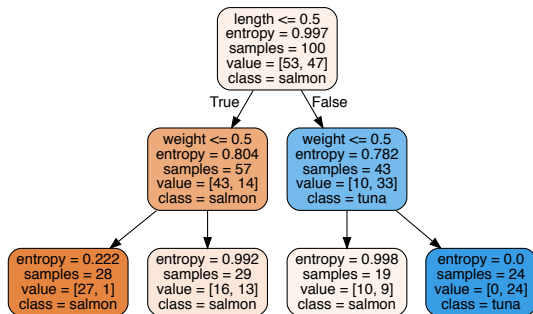
Fish dataset



CART

- ▶ Segmentation method (partitionnement récursif), **binary splits**.
- ▶ First algorithm : **CART** (classification and regression tree, Breiman, 1984) [Breiman et al., 2017]

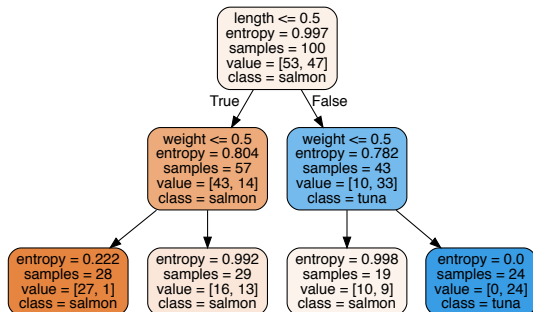
Example tree



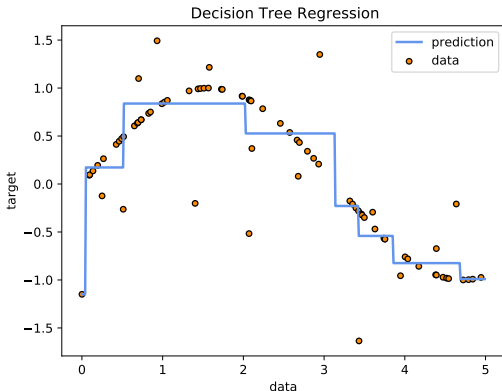
Each split should lead to more **homogeneous** nodes (in a sense that is to be formally defined)

Leaves

Leaf nodes correspond to a predicted value (class or real number for instance if $\mathcal{Y} = \mathbb{R}$)).



Decision trees can also be used for regression. In that case, the prediction is **piecewise constant**. It can be seen as a form of local averaging.



Construction of a tree

A split node corresponds to :

- ▶ a segmentation variable V
- ▶ a segmentation value / threshold if V is quantitative. If V is a class, it is a two-fold partitionning of the set of classes.

The construction of a tree requires a criterion to :

- ▶ choose the segmentation variable and value
- ▶ decide when a node is terminal (leaf node)
- ▶ associate a prediction value to all leaf nodes

Homogeneity

We want the homogeneity in the child nodes to be greater than in the parent node. Equivalently, we can define a non-negative heterogeneity function H that describes each node and that must be :

- ▶ 0 if the node is homogeneous (only one class (classification) or one output value (regression) is represented in this node).
- ▶ maximal if the values of y are uniformly distributed within the node.
- ▶ if $H(L)$ is the value of H for the left child node (and $H(R)$ for the right one), then the segmentation should minimize

$$H(L) + H(R) \tag{1}$$

Homogeneity criterion for regression

In the case of regression, $\mathcal{Y} = \mathbb{R}$ and the heterogeneity $H(n)$ of node n is its empirical variance :

$$H(n) = \frac{1}{|n|} \sum_{i \in n} (y_i - \bar{y}_n)^2 \quad (2)$$

Each y_i is a label for an input x_i that arrives in node n .

Homogeneity criterion for classification

In the case of classification, $\mathcal{Y} = [1, \dots, L]$ and several criteria exist.

Homogeneity criterion for classification : entropy (information gain)

- ▶ We use the convention that $0 \log(0) = 0$
- ▶ p_n^l is the proportion of class l in node n .

The **entropy** writes :

$$H(n) = - \sum_{l=1}^L p_n^l \log(p_n^l) \quad (3)$$

Exercise 1 : What are the maximum and minimum values of the entropy ?

Homogeneity criterion for classification : Gini impurity

The Gini impurity writes :

$$H(n) = \sum_{l=1}^L p_n^l (1 - p_n^l) \quad (4)$$

Homogeneity criterion for classification : misclassification probability

$$H(n) = 1 - \max_l (p_n^l) \quad (5)$$

If we predict the most represented class in n , then this is the misclassification probability (if we assume that the classes are indeed distributed according to the proportions).

Homogeneity criterion for classification : Gini impurity

$$H(n) = \sum_{l=1}^L p_n^l (1 - p_n^l) \quad (6)$$

Same interpretation as the previous slide (misclassification probability), but in the case we predict the classes according to the proportions in the leaves (instead of predicting the most likely class).

Prediction for a leaf node

- ▶ **regression** : predict the average value in the node
- ▶ **classification** : several possibilities
 - ▶ most represented class in the node
 - ▶ most probable class *a posteriori* if the *a priori* probabilities are known (Bayesian probabilities)
 - ▶ class that costs the less in case of missclassification

Pruning

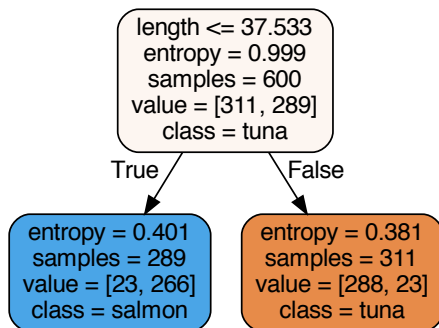
- ▶ If the segmentation is not restricted, the tree can fit the training dataset perfectly (overfitting).
- ▶ learning would then be highly dependent on the choice of the training dataset, leading to **instability**.
- ▶ To avoid it, **pruning** (élaguage) is used : it consists in restricting the size of the tree (and can be seen as an equivalent to regularization when we perform ridge regression of logistic regression).

Pre-pruning by restricting divisions

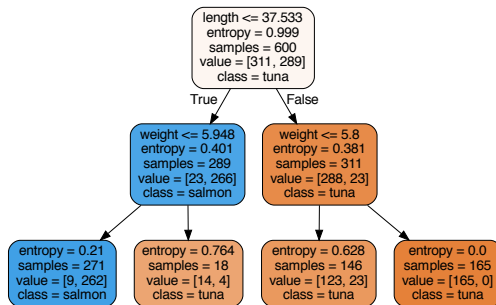
We can impose

- ▶ minimum impurity decrease
- ▶ a minimum number of samples in a leaf node
- ▶ a minimum number of samples to authorize splitting a node

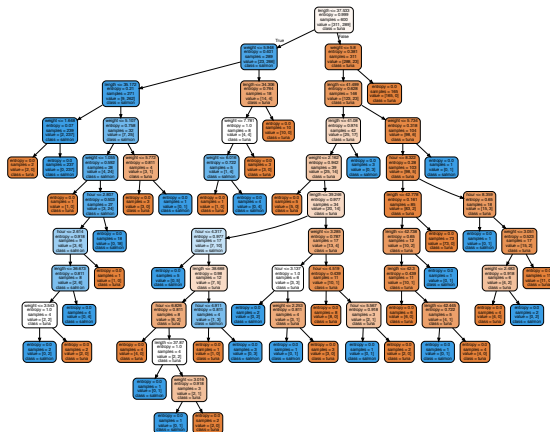
Simulation : fish dataset, max depth=1



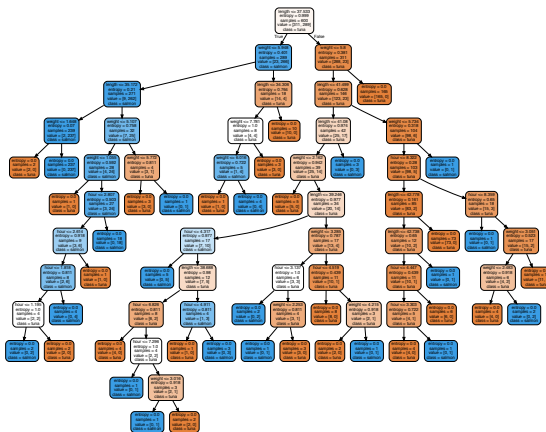
Simulation : fish dataset



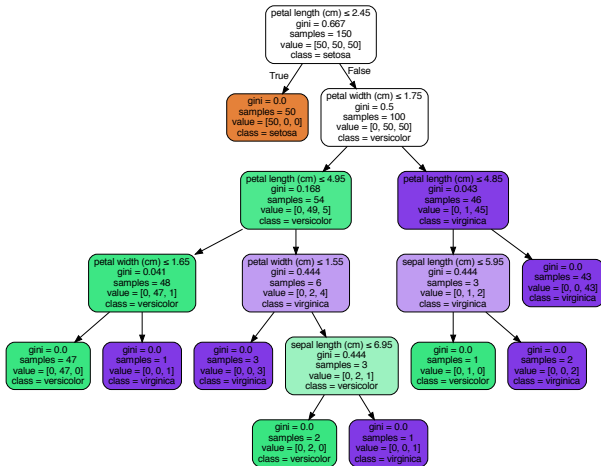
Simulation : fish dataset



Simulation : fish dataset



Overfitting : Iris dataset, max depth=34



Avoiding overfitting : Iris dataset

Exercise 2 :

Avoid overfitting by choosing a pruning method on the iris dataset
(day_5/1_cart/tree_iris.py)

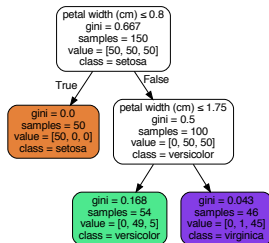


Figure – Avoid overfitting with pre-pruning.

Post-pruning

It is also possible to prune after the construction of a large tree.

Remarks on decision trees I

- ▶ Decision trees do not require hypotheses on the distribution of the features.
- ▶ **Feature selection** (determine which features matter for the quantity to predict) is integrated in the algorithm, so they might be relevant in a situation with many features (variables explicatives)
- ▶ Decision trees allow a straightforward **interpretation** of the model.

Remarks on decision trees II

- ▶ Decision trees are greedy : they might find a local minimum
- ▶ they are hierarchical : a bad choice in a segmentation at the top of the tree propagates in the whole tree
- ▶ Hence their instability and sensibility to the choice of the training set.

Decision trees

- Decision trees

- Construction of a decision tree

- Tree pruning

Ensemble learning

- Bagging

- Random forest

Ensemble learning

- ▶ *Bagging* and *boosting* are methods that combine estimators (in parallel or sequentially)
- ▶ they are often applied to decision trees
- ▶ they reach state-of-the-art performance in several supervised learning tasks [Fernández-Delgado et al., 2014] (pdf in **documents/trees/** at the root of the repo)
- ▶ they are an active area of research (both theoretically and for practical applications).

<https://scikit-learn.org/stable/modules/ensemble.html>

Aggregating

- ▶ usual setup : input space $\mathcal{X}$, output space $\mathcal{Y}$.
- ▶ we note z_b a dataset sampled from the unknown distribution ρ . If we sample b different datasets, $b \in [1, B]$,

$$z_b = \{(x_{1b}, y_{1b}), \dots (x_{nb}, y_{nb})\} \quad (7)$$

- ▶ we note $\hat{f}_{z_b}$ the estimator obtained after learning with z_b .

Aggregating : reducing the variance of the final estimator

- ▶ we note z_b a dataset sampled from the unknown distribution ρ . If we sample b different dataset, $b \in [1, B]$,

$$z_b = \{(x_{1b}, y_{1b}), \dots (x_{nb}, y_{nb})\} \quad (8)$$

- ▶ we note $\hat{f}_{z_b}$ the estimator obtained after learning with z_b .

Aggregating consists in using as an estimator :

- ▶ for regression (usual averaging)

$$\hat{f}_B = \frac{1}{B} \sum_{b=1}^B \hat{f}_{z_b} \quad (9)$$

- ▶ for classification (majority in the vote)

$$\hat{f}_B(x) = \arg \max_j |\{b, \hat{f}_{z_b}(x) = j\}| \quad (10)$$

Bootstrapping and bagging

- ▶ If B is large, it is not possible to sample B independent datasets with n samples, from a finite dataset.
Bootstrapping consists in sampling B times a sample dataset with n elements **with replacement**.
- ▶ **Bagging** is the combination of bootstrapping and aggregating.

Out-of-bag error

For an observation (x_i, y_i) , the **out-of-bag error** is the mean error on this observation among the estimators that were trained on a bootstrap dataset **not** containing it.

It is the "bootstrapping version" of the test error.

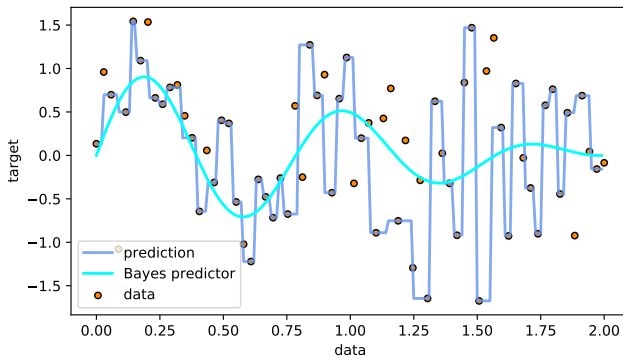
Trees and bagging

Bagging is most often used with decision trees. Several strategies are possible in order to build the base estimators $\hat{f}_{z_b}$ and to prune them.

- ▶ pre-pruning by enforcing a minimal number of samples per leaf node (e.g. 5) : good tradeoff between computation speed and quality of the prediction. The goal is to reduce high variance of the base estimators by averaging.
- ▶ pre-pruning by enforcing a maximal tree depth or maximal number of leaves.

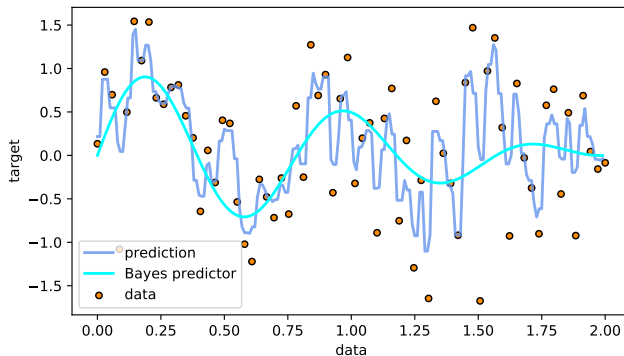
Simulation

Bagging regression
number of estimators: 1
test error: 1.22E+00
Bayes risk: 7.00E-01



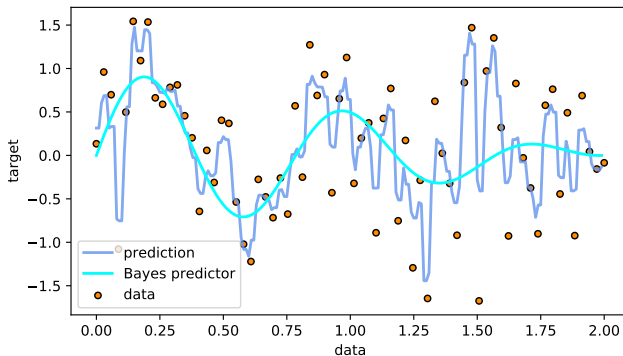
Simulation

Bagging regression
number of estimators: 10
test error: 9.09E-01
Bayes risk: 7.00E-01



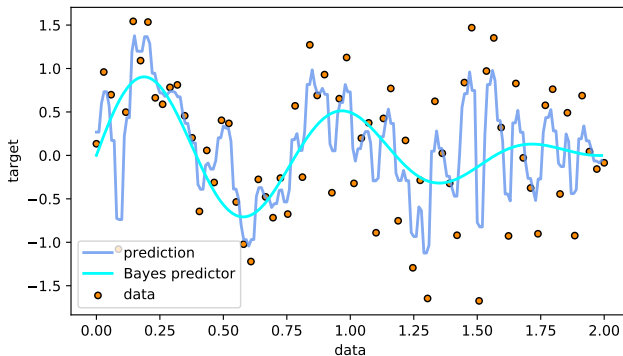
Simulation

Bagging regression
number of estimators: 20
test error: 9.39E-01
Bayes risk: 7.00E-01



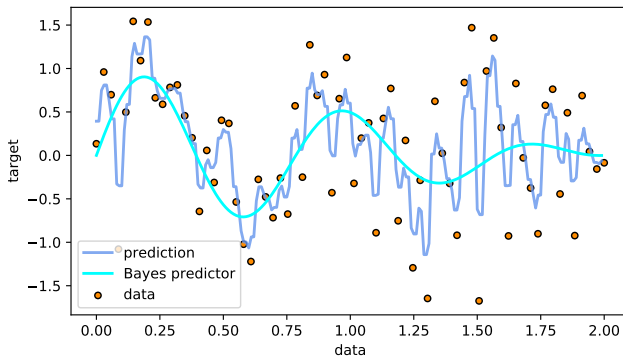
Simulation

Bagging regression
number of estimators: 50
test error: 8.95E-01
Bayes risk: 7.00E-01



Simulation

Bagging regression
number of estimators: 100
test error: $8.81\text{E-}01$
Bayes risk: $7.00\text{E-}01$



Individual estimators

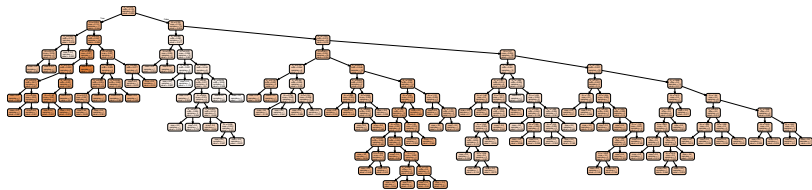


Figure – Estimator used in the averaging

Individual estimators

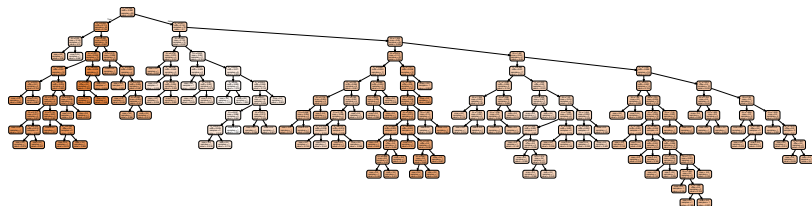


Figure – Other estimator used in the averaging

Possible issues

- ▶ number of trees to compute (B) in order to have a stable out-of-bag error.
- ▶ need for storing all the base estimators
- ▶ the final estimator is a *black-box* model.

Random forest

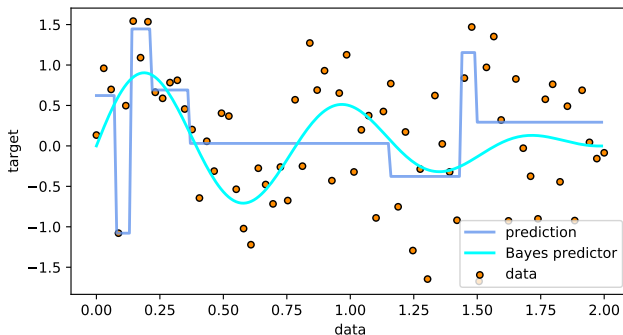
- ▶ Random forest [Breiman, 2001] is a bagging method with binary CART estimators, with additional randomness added to the choice of segmentation variables.
- ▶ the goal is to increase the **independence** between the base trees.
- ▶ although the theoretical properties of random forest are not yet fully understood, it is an important benchmark in supervised learning (but for instance not when the problem can be solved in a linear way).

Random forest II

- ▶ With random forests, it is possible to use a more brutal pruning (for instance using only trees of depth 2)
- ▶ To tune m , heuristics are used. Common ones include :
 - ▶ $m = \sqrt{d}$ for classification (d is the number of features)
 - ▶ $m = d/3$ for regression
 - ▶ cross validation.

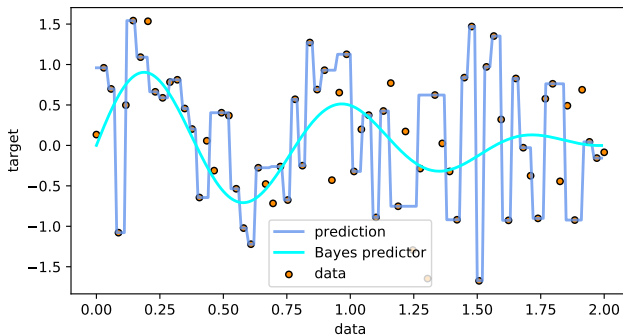
Example in 1D (1 feature, hence not really illustrative of random forest!)

Random forest regression
number of estimators: 1
max depth: 3
test error: 9.64E-01
Bayes risk: 7.00E-01



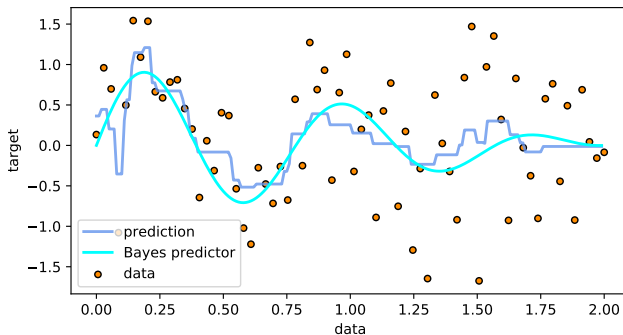
Example in 1D

Random forest regression
number of estimators: 1
max depth: 30
test error: 1.26E+00
Bayes risk: 7.00E-01



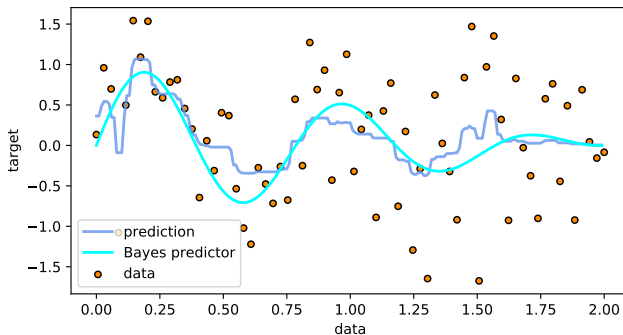
Example in 1D

Random forest regression
number of estimators: 10
max depth: 3
test error: $7.54E-01$
Bayes risk: $7.00E-01$



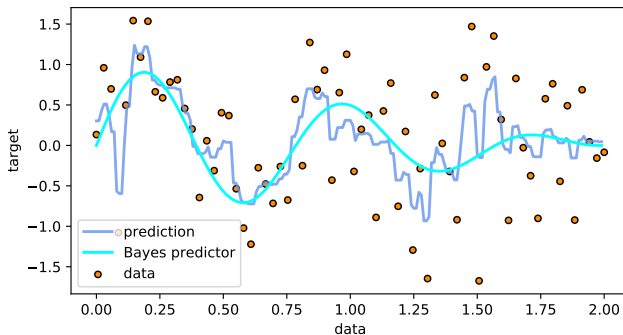
Example in 1D

Random forest regression
number of estimators: 50
max depth: 3
test error: $7.53E-01$
Bayes risk: $7.00E-01$

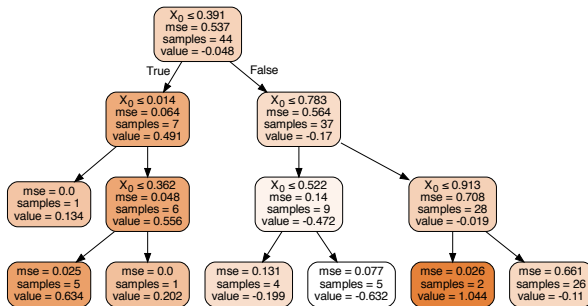


Example in 1D

Random forest regression
number of estimators: 20
max depth: 5
test error: $8.09\text{E-}01$
Bayes risk: $7.00\text{E-}01$



Estimators



Mean decrease accuracy

Although the final estimator is hard to interpret, we can still study the **importance of features** for the prediction.

We consider a feature j . For a given tree b

- ▶ compute the out-of-bag error
- ▶ shuffle the values of j in the out-of-bag sample.
- ▶ compute the new out-of-bag error
- ▶ store the decrease in prediction quality, D_j^b .

Average the D_j^b over all the estimators b in order to measure the importance of feature j .

Other hyperparameters

Many hyperparameters are involved for ensemble learning.

- ▶ **subsampling** : sampling bootstrap datasets of size $s < n$.
Smaller variance because of more independence between the trees, but higher bias.
- ▶ number of bins in each variable (for instance, impose a maximum number of bins).

See also

- ▶ boosting
- ▶ gradient tree boosting
- ▶ XGBoost : variant of gradient boosting, very efficient on several benchmarks [Chen and Guestrin, 2016]
- ▶ can exploit parallelization
- ▶ `https://xgboost.readthedocs.io/en/latest/parameter.html`
- ▶ `https://github.com/dmlc/xgboost`
- ▶ See also : Catboost `https://catboost.ai/`

References I



Breiman, L. (2001).

Random Forests.

Machine Learning, 45(1) :5–32.



Breiman, L., Friedman, J. H., Olshen, R. A., and Stone, C. J. (2017).

Classification And Regression Trees.

Routledge.



Chen, T. and Guestrin, C. (2016).

XGBoost : A scalable tree boosting system.

Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 13-17-Aug :785–794.

References II



Fernández-Delgado, M., Cernadas, E., Barro, S., and Amorim, D. (2014).

Do we need hundreds of classifiers to solve real world classification problems?

Journal of Machine Learning Research, 15 :3133–3181.